```
Transfer aborted
Paul has 550.0
Peter has -250.0
```

Although the database was left unscathed, our model objects were updated anyway. This is because Active Record wasn't keeping track of the before and after states of the various objects—in fact, it couldn't, because it had no easy way of knowing just which models were involved in the transactions.

Built-In Transactions

When we discussed parent and child tables in *Specifying Relationships in Models*, we said that Active Record takes care of saving all the dependent child rows when you save a parent row. This takes multiple SQL statement executions (one for the parent and one each for any changed or new children).

Clearly, this change should be atomic, but until now we haven't been using transactions when saving these interrelated objects. Have we been negligent?

Fortunately, no. Active Record is smart enough to wrap all the updates and inserts related to a particular `save` (and also the deletes related to a `destroy`) in a transaction; either they all succeed or no data is written permanently to the database. You need explicit transactions only when you manage multiple SQL statements yourself.

While we've covered the basics, transactions are actually very subtle. They exhibit the so-called ACID properties: they're Atomic, they ensure Consistency, they work in Isolation, and their effects are Durable (they're made permanent when the transaction is committed). It's worth finding a good database book and reading up on transactions if you plan to take a database application live.

What We Just Did